

Constraint Optimization through RTL Enhancement Using Generative AI

Team Timing Blinders

Nagarjun BV (25M1157)

Shubhanshu Shivhare (25M1173)

M.Tech, Electronic Systems, IIT Bombay — 2nd Year

Astera Labs Nebula 2026, Digital Track

Motivation

Closing timing on a block is the same manual loop, every time: read the STA report, find the path that missed, pick a fix, edit the RTL by hand, re-synthesize, check if it helped. Repeat.

- **The fix is always one of four techniques:** pipelining, register retiming, logic restructuring, or FSM re-encoding.
- **None of this loop is inherently manual** — it stays manual because nobody trusts an automated system enough to run it unsupervised.
- **We build that system**, and put the trust question first instead of last: no edit reaches the physical design stages without a formal proof that it did not change the design's behavior.

Closed-Loop Architecture

One loop, five gates, each one checked before the design is allowed to touch the next stage.

- **A lint gate** runs first, on every RTL candidate, before it is allowed anywhere near synthesis: Verilator/slang catch structural issues (unused signals, width mismatches, latch inference). Rule-matched issues get a deterministic, zero-token auto-fix; unmatched ones fall through to a small GenAI lint-repair pass. Either way, nothing reaches synthesis until lint passes.
- `run_sta` wraps OpenSTA, turns a raw timing report into ranked critical paths with slack — in a form an LLM can actually read.
- `propose_and_apply_edit` hands that path, and only that path's RTL, to the model, constrained to emit exactly one of the four techniques above as edited Verilog — never prose, never a suggestion the model then has to also implement correctly by hand. Every such edit is itself checked with EQY/SymbiYosys against the pre-edit RTL before it is accepted; fail that check and the edit is discarded, no exceptions.
- `resynth_and_check` re-synthesizes an accepted edit in Yosys and re-runs OpenSTA to confirm timing/area/power actually improved. This is a second, separate equivalence gate from the one above: it checks the *synthesized netlist* against the RTL that produced it — proving the synthesis step itself was behavior-preserving, not just the LLM's edit — and only a pass here is allowed to proceed to floorplanning.
- **A second, advisory GenAI pass** runs downstream of place-and-route (placement, CTS, routing, STA) and surfaces physical-design suggestions from the accumulated PPA trail. It observes but does not silently rewrite RTL — the two formally-gated checks above stay the only path that mutates the design.

Papers in this space (ViTAD [5], SymRTLO [4], TimingLLM [6]) get as far as diagnosing a violation and proposing a fix; none of them formally prove the fix didn't break the design before calling it a result. That

gap is where this project sits.

System	Diagnoses violation	Proposes RTL fix	Formal equiv. gate	PPA before/after
ViTAD [5] (2025)	✓	✓	×	×
SymRTLO [4] (2025)	✓	✓	×	×
TimingLLM [6]	✓	×	×	×
Nebula (this work)	✓	✓	✓	✓

Table 1: No cited system formally proves an LLM-proposed RTL edit is behavior-preserving before reporting a result — the EQY/SymbiYosys gate is what this project adds.

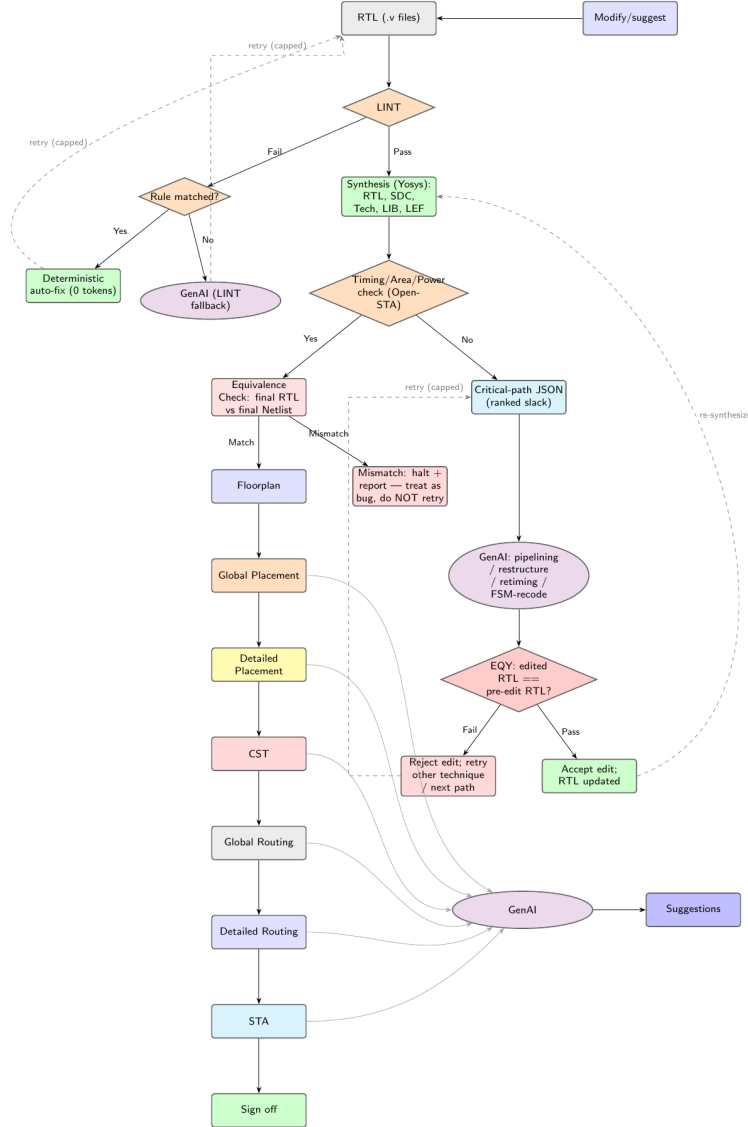


Figure 1: Closed-loop RTL-fix pipeline. Every GenAI edit is checked against pre-edit RTL before acceptance; the final equivalence check gates entry to floorplanning, and a mismatch there halts and reports rather than retrying.

Benchmark Design

The benchmark is a small SoC sized to make the loop earn its numbers, not a toy that happens to synthesize cleanly: five independent asynchronous master clock domains, generated clocks at multiple divide ratios per domain, clock-domain crossings between every peripheral and the core, roughly 50,000 standard cells post-synthesis.

A RISC-V core with split L1 instruction/data caches and a DMA engine sits on domain 1 (Clk_1), talking to the rest of the system through a shared AXI interconnect and driving generated clocks at three ratios (1.25, 1.5, 2) — the fractional ratios in particular push the divider logic past a simple counter into pulse-swallow-style generation, deliberately raising the bar on what “clock divider logic” has to mean here. Four peripherals hang off the interconnect, each in its own asynchronous domain, crossing through dedicated FIFOs rather than direct AXI taps: an FP4 dot-product accelerator on domain 2 (divide-by-2), a GPIO block on domain 3 spanning seven ratios (2, 4, 8, 16, 32, 64, 128 — seven octave steps), a timer on domain 4 (2, 4, 8), and a UART on domain 5 (2, 4, 6). Every one of those five domain boundaries is a real CDC crossing the loop has to reason about correctly when it proposes a retiming or pipelining edit near a domain edge — not a simulated one.

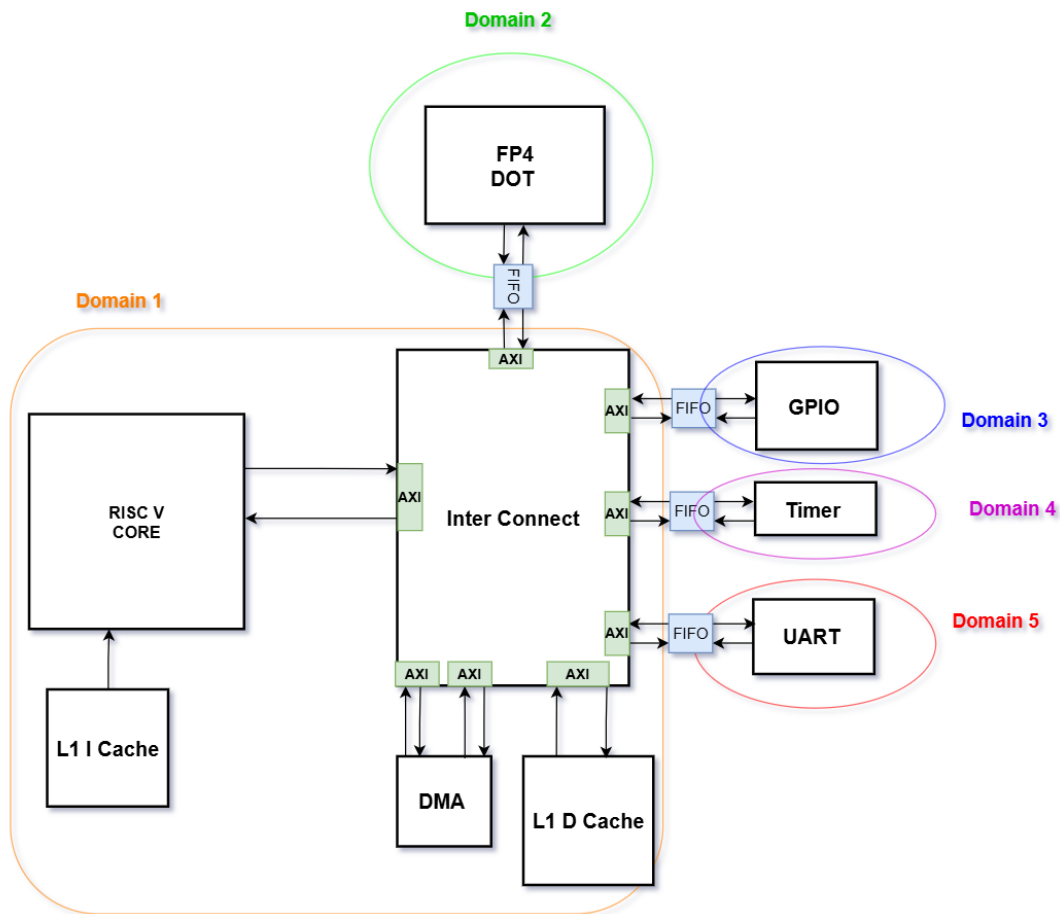


Figure 2: Nebula SoC benchmark: five asynchronous clock domains, each peripheral crossing into the AXI interconnect through a dedicated async FIFO.

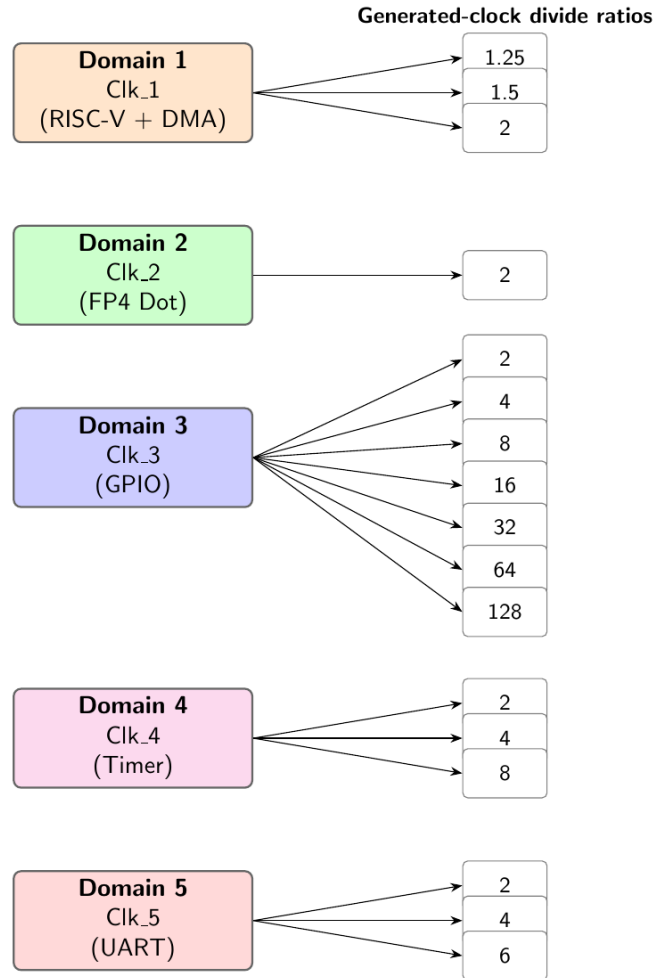


Figure 3: Generated-clock divide ratios per domain — 1.25/1.5/2 on domain 1, 2 on domain 2, seven octave steps (2 through 128) on domain 3, 2/4/8 on domain 4, 2/4/6 on domain 5.

Implementation Plan: Files and Flow

Every stage below is a concrete script against a concrete file, not an abstract box — matching the RTL→Sign-off flow diagram this abstract is built from.

1. **RTL source.** Hand-written Verilog under rtl/ — one file per block (riscv_core.v, fp4_dot.v, gpio.v, timer.v, uart.v, axi_interconnect.v, async_fifo.v, and one clock-divider module per domain, including the fractional-ratio divider for domain 1) plus one top-level nebula_soc.v that instantiates and wires them. A companion nebula_soc.sdc defines all five create_clock masters, a create_generated_clock per divider ratio, and set_clock_groups -asynchronous across every domain pair to mark the CDC boundaries for STA.
2. **Lint.** run_lint.py runs Verilator/slang against the .v sources and fails the loop early on structural issues (unused signals, width mismatches, latch inference) before spending synthesis time on broken RTL. Output: a lint log; pass/fail gates progression to synthesis.
3. **Synthesis.** run_synth.py wraps run_synth.tcl (Yosys), taking the RTL, the SDC, the sky130hd

Liberty (.lib) and LEF as input, and emitting a synthesized gate-level netlist (*_synth.v) plus an area/cell-count report.

4. **Timing/Area/Power gate.** `run_sta.py` wraps `run_sta.tcl` (OpenSTA) against the synthesized netlist and SDC, producing a structured JSON report (`artifacts/<run-id>_sta_report.json`) of ranked critical paths with slack — this JSON is the literal input the LLM reads, not a human-readable .rpt. If every path meets timing, area, and power targets, the netlist proceeds to the equivalence gate; otherwise the worst path’s JSON entry and its RTL slice go to the LLM.
5. **GenAI RTL-fix loop.** `llm_loop.py` calls the Claude API with the critical-path JSON plus the relevant RTL module, constrained by tool-use schema to return edited Verilog implementing exactly one of pipelining, logic restructuring, retiming, or FSM re-encoding. The edit is written to `artifacts/<run-id>/edit_<n>.v` and becomes the candidate RTL for the next iteration — never applied silently to the gold source.
6. **Equivalence check.** `run_eqy.py` generates a .sby config and runs EQY/SymbiYosys comparing the previous accepted RTL against the new candidate. A mismatch discards the edit and returns control to the LLM with an error report; a match promotes the candidate to the new reference RTL and re-enters the synthesis/STA gate to confirm the fix actually improved timing before it is allowed to proceed further.
7. **Floorplan through sign-off.** Once RTL is timing-clean and equivalence-proven, OpenROAD-flow-scripts takes the netlist plus LEF/DEF/tech files through floorplanning, global and detailed placement, clock tree synthesis, global and detailed routing, and final STA, on the sky130hd PDK. Each stage emits its own DEF and a timing/area/power report tagged with the run ID; the advisory GenAI pass reads across all of them and emits a suggestions document (`artifacts/<run-id>/suggestions.md`) without touching RTL. Final output: GDSII, a matched before/after PPA report, and the full equivalence-check trail, at sign-off.

Evaluation Plan

Nothing above has been run yet — this is an abstract, not a results report. What we commit to measuring once the loop runs on the benchmark:

- **Closure rate:** how many of the injected critical paths close inside a fixed iteration budget, out of the total injected.
- **Slack recovered:** on paths that do close, how much slack comes back, reported per-path, not averaged away.
- **Failures reported too:** paths that don’t converge in budget are counted and shown, not omitted.
- **Benchmark to beat:** ViTAD’s [5] 73.68% repair rate against a 54.38% plain-LLM baseline is the published number we’ll compare our actual results against once we have them — not a target we’re claiming to have already hit.

This project’s architecture, benchmark design, and evaluation plan were shaped by the following prior work.

References

- [1] *AiEDA: An Open-Source AI-Native EDA Framework*, 2024. <https://arxiv.org/html/2412.09745v1>
- [2] S. Thakur et al., *AutoChip: Automating HDL Generation Using LLM Feedback*, 2024. <https://arxiv.org/abs/2411.11856> — reference implementation: <https://github.com/shailja-thakur/AutoChip>

- [3] *Rethinking LLM-Based RTL Code Optimization Via Timing Awareness*, 2025. <https://arxiv.org/abs/2507.16808>
- [4] *SymRTLO: Enhancing RTL Code Optimization with LLMs and Neuron-Inspired Symbolic Reasoning*, 2025. <https://arxiv.org/abs/2504.10369>
- [5] *ViTAD: Timing-Violation-Aware RTL Repair with LLMs*, 2025. <https://arxiv.org/pdf/2508.13257>
- [6] *TIMINGLLM: RAG-Based FPGA Timing Closure with Large Language Models*.
- [7] *ChipSeek: RL-Guided RTL Timing Closure*, 2025. <https://arxiv.org/html/2507.04736v2>
- [8] *Closing the Loop on LLM-Generated RTL Assertions*, 2026. <https://arxiv.org/pdf/2606.21451>
- [9] *Timing-Driven Global Placement by Efficient Critical Path Extraction*, 2025. <https://arxiv.org/abs/2503.11674>
- [10] *A Machine Learning Framework for Register Placement Optimization*, 2018. <https://arxiv.org/pdf/1801.02620>
- [11] *Deep Representation Learning for Electronic Design Automation: A Survey*, 2025. <https://arxiv.org/pdf/2505.02105>
- [12] J. Yang et al., *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. <https://github.com/SWE-agent/SWE-agent>
- [13] *Automated QoR Improvement in OpenROAD with Coding Agents*, 2026. <https://arxiv.org/abs/2601.06268>
- [14] A. Allam, Y. Mansour, M. Shalan, *ASIC-Agent: A Multi-Agent System for Autonomous ASIC Design*, 2025. <https://arxiv.org/abs/2508.15940>